



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 14 - Teoría de Números

Noviembre de 2025

Caetano Borges, Nicolás Buzeta

1. Sumar Fracciones

Diseñe y escriba el algoritmo **SumarFracciones** tal que dadas $n \geq 2$ fracciones $\frac{a_1}{b_1}, \frac{a_2}{b_2}, \dots, \frac{a_n}{b_n}$, con $b_i \neq 0$ para $1 \leq i \leq n$, calcule la fracción $\frac{a}{b}$ tal que

$$\frac{a}{b} = \frac{a_1}{b_1} + \frac{a_2}{b_2} + \dots + \frac{a_n}{b_n}$$

Es importante notar que el resultado $\frac{a}{b}$ debe quedar en su mínima expresión, es decir, los valores a y b deben ser los menores posibles que representen a la fracción. Indique el tiempo de ejecución de su algoritmo (indicando también el tipo de operaciones que considera como medida de tiempo). Si lo necesita, puede invocar algoritmos estudiados en clases, indicando de manera precisa cuáles algoritmos son.

Solución

Para sumar dos fracciones, sabemos que el denominador debe ser igual. Generalmente cuando hacemos esto a mano, multiplicamos todos los denominadores juntos. Es decir:

$$\begin{aligned} \frac{a}{b} &= \frac{a_1}{b_1} + \frac{a_2}{b_2} \\ &= \frac{a_1 \cdot b_2}{b_1 \cdot b_2} + \frac{a_2 \cdot b_1}{b_2 \cdot b_1} \end{aligned}$$

Sin embargo, si realizamos el producto de todos los denominadores $b_1 \cdot b_2 \cdots b_n$, los números resultantes pueden crecer muy rápidamente, volviéndose computacionalmente costosos de manejar. El tiempo de ejecución de las operaciones aritméticas básicas (suma, multiplicación, división) depende de la cantidad de dígitos (o bits) de los operandos. Por lo tanto, para mantener la eficiencia, debemos encontrar el denominador común más pequeño posible: el **Mínimo Común Múltiplo** (mcm).

El algoritmo propuesto procede de la siguiente manera:

1. Calcular el denominador común $L = \text{mcm}(b_1, b_2, \dots, b_n)$.
2. Calcular el numerador expandido sumando los términos ajustados al nuevo denominador.
3. Simplificar la fracción resultante utilizando el **Máximo Común Divisor** (mcd).

Algoritmo SumarFracciones

Entrada: Listas de enteros $A = [a_1, \dots, a_n]$ y $B = [b_1, \dots, b_n]$ donde $b_i \neq 0$.

Salida: Enteros num, den que representan la fracción simplificada.

1. **Calcular el mcm de los denominadores:** Sea $L \leftarrow b_1$.
Para i desde 2 hasta n :

$$L \leftarrow \text{mcm}(L, b_i)$$

*Nota: Para calcular el $\text{mcm}(x, y)$ utilizamos la propiedad $\text{mcm}(x, y) = (x \cdot y) / \text{mcd}(x, y)$, invocando el **Algoritmo de Euclides** para obtener el mcd.*

2. **Calcular el numerador total:** Sea $S \leftarrow 0$.
Para i desde 1 hasta n :

$$S \leftarrow S + \left(a_i \cdot \frac{L}{b_i} \right)$$

3. **Simplificar la fracción final:** Calculamos el factor común para reducir la fracción a su mínima expresión invocando nuevamente al Algoritmo de Euclides:

$$g \leftarrow \text{mcd}(S, L)$$

4. **Retornar:**

$$\text{resultado} \leftarrow \frac{S/g}{L/g}$$

Análisis del Tiempo de Ejecución

Si consideramos como operación elemental las operaciones aritméticas con números de tamaño arbitrario (bit-complexity), el uso del mcm es crucial.

- El cálculo del mcm de n números requiere $n - 1$ llamadas al algoritmo de Euclides. El algoritmo de Euclides tiene una complejidad logarítmica respecto al valor de los operandos, $O(\log(\min(a, b)))$.
- La sumatoria realiza n multiplicaciones y divisiones.
- La simplificación final requiere 1 llamada adicional a Euclides.

Si el tamaño de los números b_i es acotado por una constante K , la complejidad estaría dominada por $O(n)$. Sin embargo, como el valor de L puede crecer, la complejidad real depende del tamaño en bits de L . El uso del mcm asegura que trabajamos con el número L más pequeño posible, minimizando el costo de las operaciones aritméticas en comparación con el producto bruto de denominadores.

2. Problema de la Raíz Discreta

Sean p y q primos distintos y sea $a \in \mathbb{Z}_p = \{0, \dots, p-1\}$. Diseñe un algoritmo eficiente que reciba como input q y a^q (mód p) y retorne a , esto es, que compute la q -ésima raíz discreta de a en módulo p . Luego, responda las siguientes preguntas:

1. ¿Cuál es la complejidad de su algoritmo?
2. ¿Qué pasa con su algoritmo si q no es primo?

Solución

Tenemos a^q (mód p), y nos gustaría obtener a (mód p), lo que significa que de alguna forma tenemos que ser capaces de modificar el exponente. No somos capaces de producir una suma en el exponente, ya que si quisieramos obtener a^{q+x} (mód p) tendríamos que hacer la operación $a^q \cdot a^x$ (mód p), pero no conocemos a así que no tenemos acceso a a^x (mód p). Lo que sí podemos hacer es producir una multiplicación en el exponente, ya que para obtener a^{qx} podemos simplemente elevar a^q (mód p) a x para obtener $(a^q)^x \equiv a^{qx}$ (mód p).

Luego, nos gustaría encontrar un x tal que $a^{qx} \equiv a^1 \equiv a$ (mód p). Para ello, necesitamos algunas nociones de Teoría de Grupos:

- Un grupo $G = (A, *)$ es una tupla cuyo primer elemento es un conjunto A y su segundo elemento es una operación $*$: $A \times A \rightarrow A$ que cumplen que
 - $*$ es asociativa
 - Existe $e \in G$ tal que $\forall g \in G, g * e = e * g = e$. A e se le llama identidad.
 - Todo elemento tiene una inversa, esto es, $\forall g \in G, \exists g^{-1} \in G$ tal que $g * g^{-1} = g^{-1} * g = e$.
- Cuando p es primo, $G = (\mathbb{Z}_p^*, \cdot)$ es un grupo, con $\mathbb{Z}_p^* = \{1, \dots, p-1\}$ y $\cdot$ la multiplicación de enteros en módulo p .
- El orden de un grupo G , denotado $|G|$, es la cantidad de elementos en el grupo. Tenemos que $|\mathbb{Z}_p^*| = p - 1$.
- El orden de un elemento g en un grupo es la menor cantidad de veces que hay que repetir la operación con ese elemento para que resulte en la identidad del grupo. Por

ejemplo, $\text{ord}(2) = 2$ en $\mathbb{Z}_3^*$, ya que $2^2 = 4 \equiv 1 \pmod{3}$.¹

- Teorema de Lagrange: Si G es un grupo finito, entonces para cualquier $g \in G$ se tiene que $\text{ord}(g) \mid |G|$, es decir, el orden de todo elemento divide al orden del grupo.

Primero que nada, notemos que $|\mathbb{Z}_p^*| = p - 1$.

Tenemos entonces que $a^{\text{ord}(a)} \equiv 1 \pmod{p}$, por lo que para cualquier $k \in \mathbb{N}$ tendremos que $a^{k \cdot \text{ord}(a)} \equiv (a^{\text{ord}(a)})^k \equiv 1^k \equiv 1 \pmod{p}$. Por el Teorema de Lagrange, $\text{ord}(a) \mid p - 1$, por lo que existe un $k \in \mathbb{Z}$ (que por lo demás es positivo, ya que tanto $\text{ord}(a)$ como $p - 1$ son positivos) tal que $k \cdot \text{ord}(a) = p - 1$. Luego, queremos lograr un exponente de la forma $(p - 1) + 1$, ya que

$$\begin{aligned} a^{(p-1)+1} &\equiv a^{p-1} \cdot a && \pmod{p} \\ &\equiv a^{k \cdot \text{ord}(a)} \cdot a && \pmod{p} \\ &\equiv 1 \cdot a && \pmod{p} \\ &\equiv a && \pmod{p} \end{aligned}$$

Con ello, necesitamos un x tal que $q \cdot x = (p - 1) + 1$. Podemos notar que aquí aparece la definición de equivalencia módulo $p - 1$:

$$\begin{aligned} q \cdot x &= (p - 1) + 1 \\ q \cdot x - 1 &= p - 1 \\ \iff q \cdot x &\equiv 1 \pmod{p - 1} \end{aligned}$$

Por lo que x es exactamente el inverso multiplicativo de q en módulo $p - 1$. Pero $p - 1$ no es primo, por lo que no todo elemento en $\mathbb{Z}_{p-1}^*$ tiene garantizada la existencia de su inverso. Sin embargo, q es primo, por lo que necesariamente $\text{gcd}(q, p - 1) = 1$, lo que garantiza la existencia del inverso multiplicativo de q en módulo $p - 1$. Para encontrar este inverso podemos usar el algoritmo extendido de Euclides.

Teniendo x , solo faltaría elevar a^q a x y tomar módulo p . Pero $(a^q)^x$ puede crecer demasiado, lo que haría que ejecutar la operación módulo sea muy costoso. Para hacer esto eficientemente, podemos usar exponenciación binaria y tomar mod p en cada iteración del loop.

El algoritmo final es:

Algorithm 1 DiscreteRoot($q, a^q \bmod p, p$)

- 1: $(d, x, t) \leftarrow \text{EuclidesExtendido}(q, p - 1)$
 - 2: $a \leftarrow \text{ExponenciaciónBinaria}(a^q, x, p)$
 - 3: **return** a
-

¹Si no existe un n tal que $g^n = 1$, entonces $\text{ord}(g) = \infty$. Esto no es muy relevante para nosotros ya que trabajaremos principalmente con grupos finitos, donde el orden de un elemento nunca es infinito.

Notemos que si

$$a^q \equiv 0 \pmod{p} \quad \text{o} \quad a^q \equiv 1 \pmod{p}$$

entonces la respuesta es 0 o 1 respectivamente, y podemos responder de manera inmediata. Podríamos agregar esto al inicio del algoritmo para ser más eficientes en estos casos, pero esto no afecta a la complejidad asintótica del algoritmo. Ahora respondemos a las preguntas del enunciado:

1. ¿Qué complejidad tiene su algoritmo?

Esta complejidad está dada por la complejidad de las subrutinas utilizadas:

- `EuclidesExtendido`($q, p - 1$) tiene complejidad $O(\log(\max\{q, p - 1\}))$.
- `ExponenciaciónBinaria`(a^q, x, p) tiene complejidad $O(\log(x))$. Como x es el inverso multiplicativo de q en módulo $p - 1$, $x \in \mathbb{Z}_{p-1}^*$, por lo que $x < p - 1$. Luego, esta subrutina tiene complejidad $O(\log(p - 1))$.

Con ello, la complejidad del algoritmo es $O(\log(p - 1))$.

2. ¿Qué pasa con su algoritmo si q no es primo?

Si q no es primo, entonces la existencia del inverso multiplicativo de q en módulo $p - 1$ no está garantizada, ya que esto depende de que $\gcd(q, p - 1) = 1$, y como $p - 1$ no es primo en general, si q no es primo esto no está garantizado. Podríamos agregar la siguiente línea entre las líneas 1 y 2 en el algoritmo para asegurar no retornar resultados erróneos:

if $d \neq 1$ **then return** `ERROR_Q_SIN_INVERSO`